

EMDW User/Programmer Guide

(under development)

February 16, 2024

Contents

1	Getting started	2
1.1	Installing <code>emdw</code>	2
1.2	A quick and easy way to start using <code>emdw</code>	2
1.3	Writing external applications linking to <code>emdw</code>	2
2	Random variables (RVs)	3
2.1	The id of an RV	3
2.2	The values that an RV can take on	3
3	Factors and their operators	4
3.1	Factors	4
3.2	Factor operations	4
3.3	Associating Factors and <code>FactorOperators</code>	4
4	Available Factors	5
4.1	<code>DiscreteTable<AssignType></code>	5
4.2	<code>SqrtMVG</code>	5
4.3	<code>AffineMVG</code>	5
4.4	<code>NonLinMVG</code>	5
4.5	<code>Wishart</code>	5
4.6	<code>ConditionalMVG</code>	6
4.7	<code>Dirichlet</code>	6
4.8	<code>Polya</code>	6
4.9	<code>DirichletSet</code>	6
4.10	<code>ConditionalPolya</code>	6
4.11	The VMP hatchery	6
4.11.1	<code>Gauss1EXPF</code>	6
4.11.2	<code>Gamma1EXPF</code>	7
4.11.3	<code>Gauss1VMP</code>	7
4.11.4	<code>DirEXPF</code>	7
4.11.5	<code>Cat1EXPF</code>	7
4.11.6	<code>Cat1VMP</code>	7
4.11.7	<code>Gauss1EXPFset</code>	7
4.11.8	<code>Gamma1EXPFset</code>	7
4.11.9	<code>CGauss1VMP</code>	7
4.11.10	<code>DirEXPFset</code>	7
4.11.11	<code>CCat1VMP</code>	7
4.11.12	<code>LinGauss1VMP</code>	7

Introduction

The purpose of this document is to get you oriented and started with using EMDW. It is currently under development – the information still is quite sketchy but should help you somewhat to find your feet.

EMDW is implemented in C++ (at least version 2020) code. In the following we give a description of the components involved, how they relate to each other, and noteworthy aspects of the code involved. In general it will also be useful to look for `.test.cc` files – these contain unit tests for the various objects and at the same time provide useful examples as to how to use them.

A graphical model is a linked graph (the PGM) of factors (also called potential functions). Each factor is defined on a number of random variables. Typically these random variables overlap with random variables used by other potential functions – that is actually quite the point of a graphical model. But lets start at the beginning.

1 Getting started

1.1 Installing emdw

To install `emdw`, follow the instructions in the `README.md` file on the `emdw` repository.

1.2 A quick and easy way to start using emdw

The easiest way to start using `emdw` is to copy and adapt an existing source file. You can use any source file that produces an executable file; however, the procedure below uses the source file `emdw/src/bin/example.cc` as example.

1. Copy and rename `emdw/src/bin/example.cc` as e.g. `emdw/src/bin/my_app.cc`.
2. Open `emdw/src/bin/CMakeLists.txt`, copy the lines

```
add_executable(example example.cc)
target_link_libraries(example emdw)
```

paste them at the bottom and edit them to be

```
add_executable(my_app my_app.cc)
target_link_libraries(my_app emdw)
```

This will associate the source file `my_app.cc` with the executable file `my_app`, and link the `emdw` library to `my_app`.

3. In the terminal, navigate to the folder `emdw/build/` and execute the command

```
cmake ../; make my_app -j4
```

This will build the library and the `my_app`. You can now execute `my_app` by navigating to `emdw/build/src/bin` and executing the command

```
./my_app
```

4. You can now change `my_app.cc` according to the requirements of your application.

1.3 Writing external applications linking to emdw

To write an external application linking to `emdw`, follow the instructions under the heading “External applications linking to EMDW” in `emdw/README.md`.

2 Random variables (RVs)

2.1 The id of an RV

These are the things that we would like to infer information about. Each RV is identified by a unique unsigned int – this should be more than general enough. To refer to it in more generic terms we use `emd::RVIdType`, defined in `emd.hpp` inside the namespace `emd`. We often use a vector of them, this is available as `emd::RVIds`. The allocation of an ID does not happen automatically – you the programmer assign it its ID. Sort of like the subscript we use symbolically when we identify a specific RV as X_7 .¹ They do not have to be consecutive. However, if they 1) are in ascending order and 2) you tell the system about this via the `presorted` parameter (in constructors etc.), setting up the graph will be slightly more efficient.

Tips for specifying / retrieving RV id numbers: Since you might use a particular RV in more than one factor (see later), you will need an efficient way of specifying and retrieving those id numbers.

In very simple applications you might want to use an enum to directly allocate RV identities. For instance:

```
enum{X,Y,Z};
```

will associate the IDs 0, 1 and 2 with the RVs X , Y and Z respectively. If you want to recall those specific IDs, you simply specify the name of the RV (such as X).

In more complicated scenarios you probably will use some incrementing counter to consecutively give IDs to RVs. However, now you will need an easy way to retrieve the IDs of particular RVs. For this we often use a `map` to record the RV IDs associated with particular aspects of the system you are modelling. The following partial code snippet gives an example:

```
RVIdType rvCnt = 0;
map<vector<size_t>, RVIdType> rvMap;

.
r = something; // eg a row index
c = anotherThing; // eg a column index
.
// record the id in a map and increment the counter
rvMap[{size_t(r),size_t(c)}] = rvCnt++;
```

2.2 The values that an RV can take on

Each RV can take on a specific set of values (*float, short, symbolic etc*) – this implicitly defines a type for the RV. Normally the specific value it takes on is unknown to us (except if we are fortunate enough to directly observe it). In `emd` we have gone to lengths to make sure that the system can use RVs of many different types. Because we are going to group them together in vectors and other structures, and C++ typically insists on a homogeneous type for the elements of its structures, this brings us to a design issue. Either we need to explicitly encapsulate the type of the RVs in an abstract class, or build a very generic type that can take on any value we please (which of course will internally do similar abstract class magic). This is what we opted for – the class is called `AnyType` and you can literally assign anything to it². However, in order to extract the value from it again, you need to use a type conversion operator; i.e., you need to know what is in the specific `AnyType` variable. Since this typically happens inside factor classes (see below), this ought not be a problem. Carefully work through `anytype.hpp` and make sure that you understand how to use it. For example:

```
AnyType x;
x = int(7); // put an integer in it
x = (string("anytype")); // no, lets replace it with a string
string retrieved = string(x); // and retrieve the string
```

¹We could of course assign the IDs automatically, but experience says that reasoning about the task at hand is easier if you explicitly control the IDs.

²I did consider Boost's similar class, but ultimately the type conversion in our version was easier to use

Important: When you stick a value into an `AnyType`, make very sure that it goes in there as the specific RV value type that your factor is defined for. If, for instance, your factor is defined on `float` values, and you by accident stick a `double` value into it, the type conversion when you want to retrieve its actual type and value again, will fail. **Be especially wary of using unspecified integer constants, they revert to `int` and *not* to `unsigned`.**

In `emdwl.hpp` there is a predefined name for the type corresponding to a vector of RV values, namely `emdwl::RVVals`.

3 Factors and their operators

3.1 Factors

You can think of a factor as an extension of the good old-fashioned probability distribution. They are positive, but need not be normalised to unity area/sum. In our PGMs we want to use various types of them, therefore we created `Factor` (from `factor.hpp,tcc,cc`) as the abstract base class. To facilitate using a mix of factor types, we almost always contain them in a shared pointer or a unique pointer – these are found in `emdwl.hpp` as `rcptr` and `uniptr`.

3.2 Factor operations

Factors get operated on by `FactorOperators` to result in other `Factors` (which might be of the same or a different type). You will find the following pure virtual functions in the `Factor` class (in `factor.hpp`). On a deeper level the “dynamic double dispatch” mechanism (see below) is used to link them up to particular `FactorOperators` (from `factoroperator.hpp`):

- `absorb` and `inplaceAbsorb`: Factor products. Essential.
- `cancel` and `inplaceCancel`: Factor division. This is used by the belief update algorithms.
- `marginalize`: Factor reduction via RV marginalisation. Essential.
- `observeAndReduce`: Factor reduction via RV observation. The observed variables are eliminated from the resulting factor by summing / integrating over them.
- `inplaceDampen`: Factor damping – smoothing/stabilising messages by adding weighted factors together. Loopy message passing can be unstable and message damping is used to lessen the chances of this.
- `normalize` and `inplaceNormalize`: Factor normalisation – getting it back to unity volume. Essential.
- `sample`: Draws random samples from the particular factor. Amongst others useful for Gibbs-sampling based inference.

3.3 Associating Factors and FactorOperators

This section concerns how we make the association between `FactorOperators` and `Factors`. Unfortunately, it is somewhat involved. If after reading the following you are still confused, simply follow the examples you’ll find in the various `.test.cc` unit tests. You will soon get used to how to use it, and for the time being that should be enough.

Multiple operations applies to a specific factor type, and the same operation might apply to more than one factor type. How to know which to associate with which? We can either make extensive use of dynamic casts (which we unfortunately do have to use sometimes), or we can use a technique called *dynamic double dispatch* (DDD). We opted (mostly) for the latter. You quite likely will make use of code that uses DDD, although it is unlikely that you will change or add to it. From your point of view you can mostly think of this as boiler-plate code that you simply make use of. To understand more about this:

- See url <http://www.drdobbs.com/cpp/message-handling-without-dependencies/184429055>
- In `factor.hpp,tcc` have a look at the `dynamicApply` and `dynamicInplaceApply` members.
- In `factoroperator.hpp,tcc` have a look at the `process` and `inplaceProcess` members. Also note the two-level definition of a factor operator, the bottom with templates and the top without.
- Then inspect a specific factor implementation like `DiscreteTable` (`discretetable.hpp,tcc`) to see how all of this is tied together.

Mostly this should not concern you too much. The important point is that you can define and specify several versions of `FactorOperators`. You can see this in action in cases where, for instance, we want to replace sum-marginalisation with max-marginalisation.

4 Available Factors

4.1 DiscreteTable<AssignType>

This is *the* generic discrete factor, derived from abstract class `Categorical`. Because it uses a sparse implementation, rather large `DiscreteTable`'s are feasible. Have a look at its unit tests in `discretetable.test.cc` to see how it is used. The `AssignType` can be any *single* discrete type³ such as `int`, `unsigned`, `string`, etc. Once again, take note of the warning at the bottom of Section 2.2. If you want to use an integer type it might be better to go for `int` in preference to the other options.

4.2 SqrtMVG

This class implements multi-variate Gaussians in a numerically sound manner. It makes use of the canonical form, but the precision matrix $\mathbf{K}$ is now replaced by a lower triangular matrix $\mathbf{L}$ where $\mathbf{K} = \mathbf{L}\mathbf{L}^T$. I.e. $\mathbf{L}^{-1}$ is a multivariate version of the standard deviation whereas $\mathbf{K}^{-1}$ is a multivariate version of the variance. A possible downside is that it inherently can not represent a Gaussian with negative variances – an ugly animal that Minka explicitly makes use of. By making extensive use of Givens rotations and rank-one updates, `SqrtMVG` studiously avoids ever multiplying out the square again.

It also provide stubs useful for building more intricate classes on top of this. Specifically, the `constructAffineGaussian` static member function facilitates building affine Gaussian joints (as is used in Kalman filters), and the `constructFromSigmaPoints` static member uses sigma points to construct non-linear Gaussian joints (as is used in the unscented Kalman filter).

4.3 AffineMVG

An affine transformation $\mathbf{Y} = \mathbf{A}\mathbf{X} + \mathbf{c} + \text{noise}$ where both $\mathbf{X}$ and noise are Gaussian, $\mathbf{A}$ is a matrix and $\mathbf{c}$ is a fixed offset vector. It turns out that both $\mathbf{Y}$, as well as $\mathbf{X}, \mathbf{Y}$ are both Gaussian. This is typically used in Kalman filters and builds on `SqrtMVG`.

4.4 NonLinMVG

NIY

4.5 Wishart

NIY

³But stay away from `bool`. In the C++ language standard `std::vector<bool>` is being handled differently from other vectors. Even to the regret of some standards committee members, this is causing untold grief; also here. If you want `bool`, rather use `Bool` which we define in `oddsandsods.hpp`.

4.6 ConditionalMVG

NIY

4.7 Dirichlet

The Dirichlet distribution - this is the conjugate prior to a categorical distribution. More or less equivalent to `DirEXPF` (see below).

4.8 Polya

A categorical conditioned on a Dirichlet, i.e. $p(\mathbf{X}|\boldsymbol{\theta})$, forming a $p(\mathbf{X}, \boldsymbol{\theta})$ joint after the Dirichlet has been multiplied in. Similar (but not equivalent) to `Cat1VMP` below.

4.9 DirichletSet

A set of Dirichlets. More or less equivalent to `DirEXPFset` (see below).

4.10 ConditionalPolya

Based on a conditioning categorical Z we get different Polya's $p(W|Z, \boldsymbol{\Theta})$ where the distribution of categorical W is based on $\boldsymbol{\theta}_k$ with $Z = k$. Forms a core component of an LDA system. Similar (but not equivalent) to `CCat1VMP` below.

4.11 The VMP hatchery

We are also developing several classes that employs variational message passing (VMP) type message passing. These still need some more decoration before passing as fully-fledged EMDW factors. Until then they are safely parked in the `src/vmp` directory.

The following file naming convention is used:

<name>expf.hpp: The "expf" bit indicates that the distribution is in exponential form. Go and check out wiki or Bishop's book if you are uncertain what this is and why it matters. These distributions mostly are used as priors or building blocks for the VMP distributions (see below). In some cases there will be a "1" directly before this i.e. "1expf". The "1" simply indicates that this is a one-dimensional distribution. If the "1" is omitted the implication is that it is a multi-dimensional distribution.

<name>vmp.hpp: These are conditional distributions which support variational message passing. Variational message passing is very useful when the required marginalisations do not have exact conjugate forms and therefore need approximation - go and read John Winn's article for more details. VMP classes typically have member functions that can send/receive messages from parent or child nodes. Messages from a parent updates the random variable(s) in the condition part of the distribution, whereas messages from a child updates the variables to the left of the condition sign. When a "1" is present in the class name, it means that there is only one (scalar) variable to the left of the condition sign.

Conditioning on a categorical: Often a VMP distribution is conditioned on a categorical variable. This implies that for each value of that categorical, we get a different distribution for the variables to the left of the condition sign. In these cases the VMP file <name> starts with a "c", followed by the name of the individual distributions. Typically marginalising out that categorical variable results in a type of mixture distribution. In that case the various base components of that mixture are collected in a <name>expfset.hpp file.

4.11.1 Gauss1EXPF

One-dimensional Gaussian. The one-dimensional version of `SqrtMVG`.

4.11.2 `Gamma1EXPF`

A Gamma distribution - the one dimensional version of the Wishart distribution. We prefer the same degrees-of-freedom (DOF)-based parameterisation as is also used in the Wishart.

4.11.3 `Gauss1VMP`

A VMP one-dimensional Gaussian with a `Gauss1EXPF` parent as prior on its mean, and a `Gamma1EXPF` as prior on its precision.

4.11.4 `DirEXPF`

Dirichlet distribution.

4.11.5 `Cat1EXPF`

A one-dimensional categorical. It readily interfaces with `DiscreteTable` and is the one-dimensional version of that.

4.11.6 `Cat1VMP`

A VMP one-dimensional categorical with a `DirEXPF` parent. Can marginalise to a child message `Cat1EXPF`.

4.11.7 `Gauss1EXPFset`

A set of one-dimensional Gaussians.

4.11.8 `Gamma1EXPFset`

A set of Gamma distributions.

4.11.9 `CGauss1VMP`

A conditional one-dimensional Gaussian (i.e. a mixture), conditioned on a `Cat1EXPF` parent, a `Gauss1EXPFset` as the parent containing the priors for the means, and a `Gamma1EXPFset` parent containing the priors for its precisions.

4.11.10 `DirEXPFset`

A set of Dirichlet distributions.

4.11.11 `CCat1VMP`

A VMP one-dimensional conditional categorical, conditioned on a `Cat1EXPF` parent with `DirEXPFset` as a prior for the various categoricals.

4.11.12 `LinGauss1VMP`

A VMP affine transformation of a `Gauss1EXPF` as parent. This is identical to a `Gauss1VMP` which somehow has a fixed value for its precision parameter.

5 Linking them up: the probabilistic graphical model (PGM)

Although there are very specific requirements for how factors are correctly linked up, in `emdw` we are usually not concerned with that detail since it is handled automatically. The way they are linked up, however, will depend on the type of graphical model structure we are considering. These include region graphs, cluster graphs and factor graphs. In the current version of the software, we have only implemented

cluster graphs and also factor graphs as a topological specialisation of cluster graph. Because factor graphs are handled as a special type of cluster graph, the same inference routines are applicable to both.

The exact composition of the components that a PGM is made up of, depends a bit on the inference algorithm that is going to run on it – it has to save the information that the inference algorithm is depending on.

5.1 Cluster graphs and loopy message passing

We assemble the cluster graph itself in an object (predictably) called `ClusterGraph`, which is available from `clustergraph.hpp`. To do probabilistic inference on this we currently have two options available:

- Loopy belief *propagation* – the Schafer-Shenoy algorithm, and
- Loopy belief *update* – the Lauritzen-Spiegelhalter algorithm (also known as the *absorption algorithm*).

Both of these make use of message damping (mentioned earlier when we discussed `inplaceDamping`). In particular, note the numeric value of the damping factor. Its valid assignment range is $[0, 1)$. A damping factor of 0 means no smoothing while higher values indicate a higher degree of smoothing. Typical values are around 0.25 – it is unusual (and not recommended) to use damping values bigger than 0.5.

5.2 Loopy belief *propagation*:

In this approach we need to keep record of each cluster’s internal factor, as well as the message factors arriving at the cluster. In `lbp_cg.hpp` you will find the routines to do probabilistic inference on this graph:

- `loopyBP_CG` does the loopy belief propagation, and after convergence
- `queryLBP_CG` can answer probability queries.

5.3 Loopy belief *update*:

Instead of the combination of internal factors and message factors, we can also redo the inference to function i.t.o. cluster beliefs and sepset beliefs. For this setup we have to be able to do factor division. It has some very nice properties and is relevant for expectation propagation. However, sometimes (especially with the Gaussian classes), the division operator can cause instabilities. See Koller and Friedman Section ... for more details.

For this we use the same `ClusterGraph` structure, but now with different inference routines (from `lbu_cg.hpp`):

- `loopyBU_CG` does the loopy belief propagation, and after convergence
- `queryLBU_CG` can answer probability queries.

6 Examples

6.1 Example 1: using RVs and factor operators

An example that illustrates how to define and use discrete RVs, factors and factor operators can be found at `emd/src/bin/example.cc`. The comments in this source file explain the code quite well; the discussion here is meant to provide a high-level overview of the ideas and their implementation in code. The discussion assumes reasonable familiarity with basic C++ concepts, including C++ Standard Library containers, classes and objects (including inheritance), smart pointers (both shared and unique pointers), memory management in C++, and templates.

We usually start the implementation of a PGM by predefining some types and constants, which are done in the following lines.

```

61:   typedef int T;
62:   typedef DiscreteTable<T> DT;
63:   double defProb = 0.0;
64:   rcptr< vector<T> > binDom (
65:       new vector<T>{0,1});

```

Line 61 defines the shorthand type `T` for the values that the RVs can take on, and line 62 defines the shorthand `DT` for `DiscreteTable<int>` (see Subsection 4.1). Line 63 defines the default probability to use in the discrete factor for all cases where an explicit probability has not been specified. Lines 64-65 define the domain of the binary RVs used in this example; this is done by creating a vector containing values 0 and 1, and referring to this vector using the shared pointer `binDom`.

This example uses two binary RVs, X and Y , which are defined as follows (also see Subsection 2.1):

```

78:   enum{X, Y};

```

Next, the example illustrates how the following probability distribution, $p(X, Y)$, is created as a factor:

X	Y	$p(X, Y)$
0	0	0.0
0	1	0.5
1	0	0.5
1	1	0.0

The simplest way to create this factor is to directly declare a `DiscreteTable<int>` object as shown below.

```

99:   DT objXY(
100:       {X,Y},
103:       {binDom,binDom},
104:       defProb,
105:       {
106:           {{0,1}, 0.5},
107:           {{1,0}, 0.5},
108:       } );

```

In this statement, the object `objXY` of type `DT` (i.e., `DiscreteTable<int>`) is created, containing variables X and Y (line 100), both having a binary domain (line 103), with the default factor entry being 0.0 (line 104), and the combinations, $(X = 0, Y = 1)$ and $(X = 1, Y = 0)$ assigned factor values of 0.5 (lines 105-108) and the remaining combinations assigned the default value of 0.0.

The direct object declaration, however, is not very useful for most problems, since we then need to know the object type and work with the object variable. A much more flexible approach is to refer to the factor using a smart pointer to an abstract `Factor` object. The creation of the same factor using this dynamic object creation is shown below.

```

134:   rcptr<Factor> ptrXY =
135:       uniptr<DT>(<
136:           new DT(
137:               {X,Y},
138:               {binDom,binDom},
139:               defProb,
140:               {
141:                   {{0,1}, 0.5},
142:                   {{1,0}, 0.5},
143:               } ));

```

The `new` command (line 136) creates a (raw) c pointer to a `DT` object, which is then transferred to a temporary (smart) unique pointer to a `DT` object (line 135), which is then transferred to a (smart) shared pointer to an abstract `Factor` object (line 134), called `ptrXY`. For more detail discussion about dynamic object creation, see the comments in the source file.

The rest of the example illustrates the use of factor operators, applied to the factor $p(X, Y)$. To “marginalise out” X – i.e., to calculate $p(Y) = \sum_X p(X, Y)$ – we can use the following statement:

```
190:    rcptr<Factor> ptrY = ptrXY->marginalize({Y});
```

Here, the member function `marginalize` of the `Factor` object to which `ptrXY` points is called, using the argument `{Y}` that specifies the RVs to *retain*. The member function `marginalize` returns a unique pointer to a `Factor` object, which is transferred to the shared pointer `ptrY`.

To perform factor division – for example, to calculate the conditional distribution $p(X|Y) = p(X, Y)/p(Y)$ – we can use the following statement:

```
198:    rcptr<Factor> ptrXgY = ptrXY->cancel(ptrY);
```

Here, the member function `cancel` is called with the argument `ptrY`, which points to the factor to divide by, and the result is again a pointer to `Factor` object.

To multiply factors, we can use the `absorb` member function as shown below:

```
205:    std::cout << __FILE__ << __LINE__ << ": " << *ptrXgY->absorb(ptrY) << std::endl;
```

Here, $p(X|Y)$ is multiplied with $p(Y)$, which results in $p(X, Y)$. In this case, the resulting factor is immediately written to the console.

The factor operations can be strung together as illustrated below:

```
212:    std::cout << __FILE__ << __LINE__ << ": " << *ptrXY->cancel(ptrY)->absorb(ptrY)
->normalize() << std::endl;
```

Here, $p(X, Y)$ is first divided by $p(Y)$, then multiplied by $p(Y)$, after which the result is normalised and written out to the console.

Lastly, known RV values can be incorporated by using the `observeAndReduce` member function. For example, if it is known that $X = 0$, then we can use the following statement to incorporate this information into $p(X, Y)$:

```
218:    std::cout << __FILE__ << __LINE__ << ": " << *ptrXY->observeAndReduce({X},{0})
->normalize() << std::endl;
```

Here, $X = 0$ is first inserted into $p(X, Y)$, resulting in $p(X = 0, Y)$, after which it is normalised, resulting in $p(Y|X = 0)$ ⁴. Note that when the evidence $X = 0$ is incorporated into the factor, it essentially removes X as an RV (the resulting factor is only a function of Y), hence the term “reduce”.

Contributors

The following people contributed towards developing this document:
Johan du Preez, Corné van Daalen...

⁴The normalisation is done to ensure that the resulting factor is a valid distribution i.t.o. Y .